

Bases de Dados

SQL Avançado



Sumário

1

Funções, Gatilhos

2

Transações

Bibliografia:

– Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. – Belo, O., “Bases de Dados Relacionais: Implementação com MySQL”, FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5.

SQL Avançado

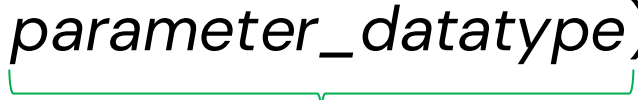
➔ Funções

O que é?

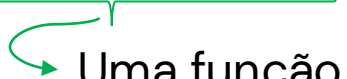
Uma função é um bloco de código SQL semelhante a uma stored procedure, mas que **obrigatoriamente retorna um valor** e pode ser usada diretamente dentro de uma query. É como uma função matemática: dás um input, recebes um output.

Sintaxe Básica

DELIMITER &&

CREATE FUNCTION <function_name> ( )

RETURNS  [NOT] DETERMINISTIC

BEGIN  

Body_section → é preciso especificar pelo menos uma instrução RETURN.

END &&

DELIMITER;

Para usar: SELECT function_name(valor);

SQL Avançado

➔ Funções

Function vs Stored Procedure

	Function	Stored Procedure
Retorna valor	Obrigatório	Opcional (OUT)
Usada em queries	SELECT, WHERE	Só com CALL
Parâmetros OUT	Não	Sim
Pode fazer DML	Limitado	Sim

SQL Avançado

➔ Funções

Exemplo

Calcular o valor total pago por um cliente:

```
DELIMITER $$

CREATE FUNCTION fn_total_pago(p_customer_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE v_total DECIMAL(10,2);

    SELECT SUM(amount) INTO v_total
    FROM payment
    WHERE customer_id = p_customer_id;

    RETURN COALESCE(v_total, 0);
END$$

DELIMITER ;
```

```
-- Usar diretamente numa query
SELECT first_name, last_name, fn_total_pago(customer_id) AS total_pago
FROM customer
LIMIT 10;
```


SQL Avançado

➔ Funções

Exemplo

Classificar um cliente com base nos seus alugueres:

```
DELIMITER $$

CREATE FUNCTION fn_classificar_cliente(p_customer_id INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE v_total INT;

    SELECT COUNT(*) INTO v_total
    FROM rental
    WHERE customer_id = p_customer_id;

    IF v_total > 30 THEN
        RETURN 'VIP';
    ELSEIF v_total > 10 THEN
        RETURN 'Regular';
    ELSE
        RETURN 'Ocasional';
    END IF;
END$$

DELIMITER ;
```

```
SELECT first_name, last_name, fn_classificar_cliente(customer_id) AS tipo
FROM customer;
```

SQL Avançado

➔ Funções

DETERMINISTIC vs NOT DETERMINISTIC

```
-- DETERMINISTIC: o mesmo input dá sempre o mesmo output
CREATE FUNCTION fn_desconto(p_rate DECIMAL(4,2))
RETURNS DECIMAL(4,2)
DETERMINISTIC ...
```

```
-- NOT DETERMINISTIC: o output pode variar (ex: usa NOW() ou dados da tabela)
```

```
CREATE FUNCTION fn_dias_desde_aluguer(p_rental_id INT)
RETURNS INT
NOT DETERMINISTIC ...
```

SQL Avançado

➔ Funções

Alterar e Eliminar

```
-- Eliminar
DROP FUNCTION fn_total_pago;

-- Ver funções existentes
SHOW FUNCTION STATUS WHERE Db = 'sakila';

-- Ver código de uma função
SHOW CREATE FUNCTION fn_total_pago;
```


A Linguagem SQL

➔ Funções

Alterar e Eliminar

```
-- Eliminar
DROP FUNCTION fn_total_pago;

-- Ver funções existentes
SHOW FUNCTION STATUS WHERE Db = 'sakila';

-- Ver código de uma função
SHOW CREATE FUNCTION fn_total_pago;
```

Resumo: Uma função retorna **sempre um valor** e pode ser usada diretamente em queries. É ideal para **cálculos reutilizáveis** e **lógica de classificação**. Ao contrário das procedures, são chamadas com `SELECT` e não com `CALL`.

SQL Avançado

➔ Gatilho (Trigger)

O que é?

Um trigger é um bloco de código SQL que é **executado automaticamente** quando ocorre um evento numa tabela (INSERT, UPDATE ou DELETE).

É como um alarme: defines as condições e ele dispara sozinho quando essas condições se verificam.

SQL Avançado

➔ Gatilho (Trigger)

Sintaxe Básica

```
DELIMITER $$

CREATE TRIGGER nome_trigger
BEFORE | AFTER INSERT | UPDATE | DELETE
ON tabela
FOR EACH ROW
BEGIN
    -- código SQL aqui
END$$

DELIMITER ;
```

SQL Avançado

➔ Gatilho (Trigger)

Tipos

Momento	Evento	Descrição
BEFORE	INSERT	Antes de inserir um registro
AFTER	INSERT	Depois de inserir um registro
BEFORE	UPDATE	Antes de atualizar um registro
AFTER	UPDATE	Depois de atualizar um registro
BEFORE	DELETE	Antes de eliminar um registro
AFTER	DELETE	Depois de eliminar um registro

SQL Avançado

➔ Gatilho (Trigger)

OLD e NEW

Dentro de um trigger tens acesso aos valores **antes** e **depois** da alteração:

	INSERT	UPDATE	DELETE
NEW	Novo registo	Valor novo	✗
OLD	✗	Valor antigo	Registo eliminado

-- Exemplo

SET NEW.coluna = UPPER(NEW.coluna); -- altera o valor antes de guardar

SQL Avançado

➔ Gatilho (Trigger)

Exemplo

Garantir que o rental_rate de um filme nunca é negativo:

```
DELIMITER $$

CREATE TRIGGER trg_film_check_rate
BEFORE INSERT ON film
FOR EACH ROW
BEGIN
    IF NEW.rental_rate < 0 THEN
        SET NEW.rental_rate = 0;
    END IF;
END$$

DELIMITER ;
```

SQL Avançado

➔ Gatilho (Trigger)

Exemplo

Registrar numa tabela de log sempre que um pagamento é inserido:

```
-- Criar tabela de log primeiro
CREATE TABLE payment_log (
    log_id      INT AUTO_INCREMENT PRIMARY KEY,
    payment_id  INT,
    amount      DECIMAL(5,2),
    log_date    DATETIME
);
```

```
DELIMITER $$
```

```
CREATE TRIGGER trg_payment_log
AFTER INSERT ON payment
FOR EACH ROW
BEGIN
    INSERT INTO payment_log (payment_id, amount, log_date)
    VALUES (NEW.payment_id, NEW.amount, NOW());
END$$
```

```
DELIMITER ;
```

SQL Avançado

➔ Gatilho (Trigger)

Exemplo

Guardar o valor antigo do rental_rate antes de ser alterado:

```
CREATE TABLE film_price_history (  
    film_id          INT,  
    old_rate         DECIMAL(4,2),  
    new_rate         DECIMAL(4,2),  
    changed_at       DATETIME  
);
```

```
DELIMITER $$
```

```
CREATE TRIGGER trg_film_price_history  
BEFORE UPDATE ON film  
FOR EACH ROW  
BEGIN  
    IF OLD.rental_rate <> NEW.rental_rate THEN  
        INSERT INTO film_price_history  
        VALUES (OLD.film_id, OLD.rental_rate, NEW.rental_rate, NOW());  
    END IF;  
END$$
```

```
DELIMITER ;
```


SQL Avançado

➔ Gatilho (Trigger)

Alterar e Eliminar

```
-- Eliminar  
DROP TRIGGER trg_payment_log;
```

```
-- Ver triggers existentes  
SHOW TRIGGERS FROM sakila;
```

```
-- Ver triggers de uma tabela específica  
SHOW TRIGGERS FROM sakila LIKE 'payment';
```

SQL Avançado

➔ Transações

O que é?

Uma transação é um conjunto de operações SQL que são executadas como uma unidade: ou todas têm sucesso, ou nenhuma é aplicada.

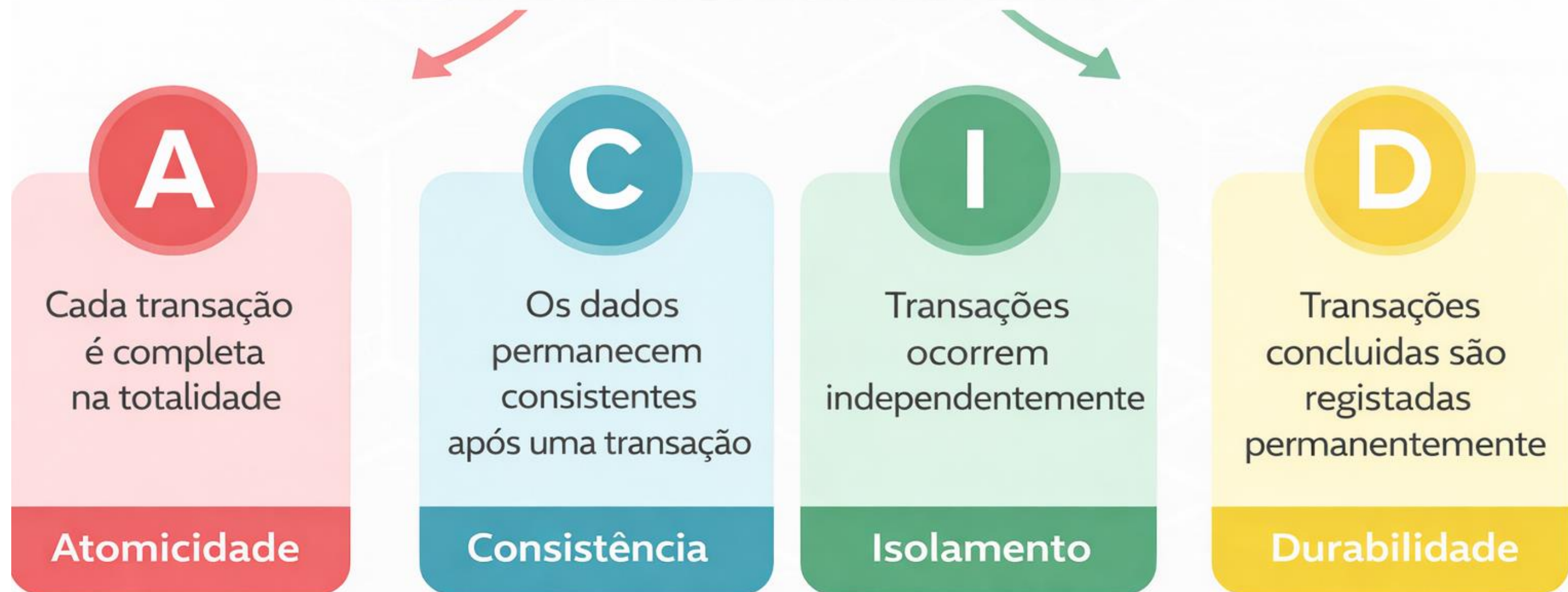
Analogia: É como uma transferência bancária, o dinheiro só sai de uma conta se entrar na outra. Se algo falhar, tudo reverte.

A transação permite executar um conjunto de operações para garantir que a BD nunca contém o resultado de operações parciais.

SQL Avançado

➔ Transações

Propriedades ACID em SGBD



SQL Avançado

➔ Transações

- Para iniciar uma transação, usa-se a instrução **START TRANSACTION**.
- Para confirmar a transação atual e tornar as suas alterações permanentes, usa-se a instrução **COMMIT**.
- Para reverter a transação atual e cancelar as suas alterações, usa-se a instrução **ROLLBACK**.

```
START TRANSACTION;
```

```
-- operações SQL aqui
```

```
COMMIT; -- confirma e aplica tudo
```

```
--ou
```

```
ROLLBACK; -- cancela e reverte tudo
```

SQL Avançado

➔ Transações

Exemplo

Registar um aluguer e o respetivo pagamento – as duas operações devem ser atómicas:

```
START TRANSACTION;  
  
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)  
VALUES (NOW(), 1, 1, 1);  
  
INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)  
VALUES (1, 1, LAST_INSERT_ID(), 4.99, NOW());  
  
COMMIT;
```

Se algo falhar antes do COMMIT, nenhuma das inserções é guardada.

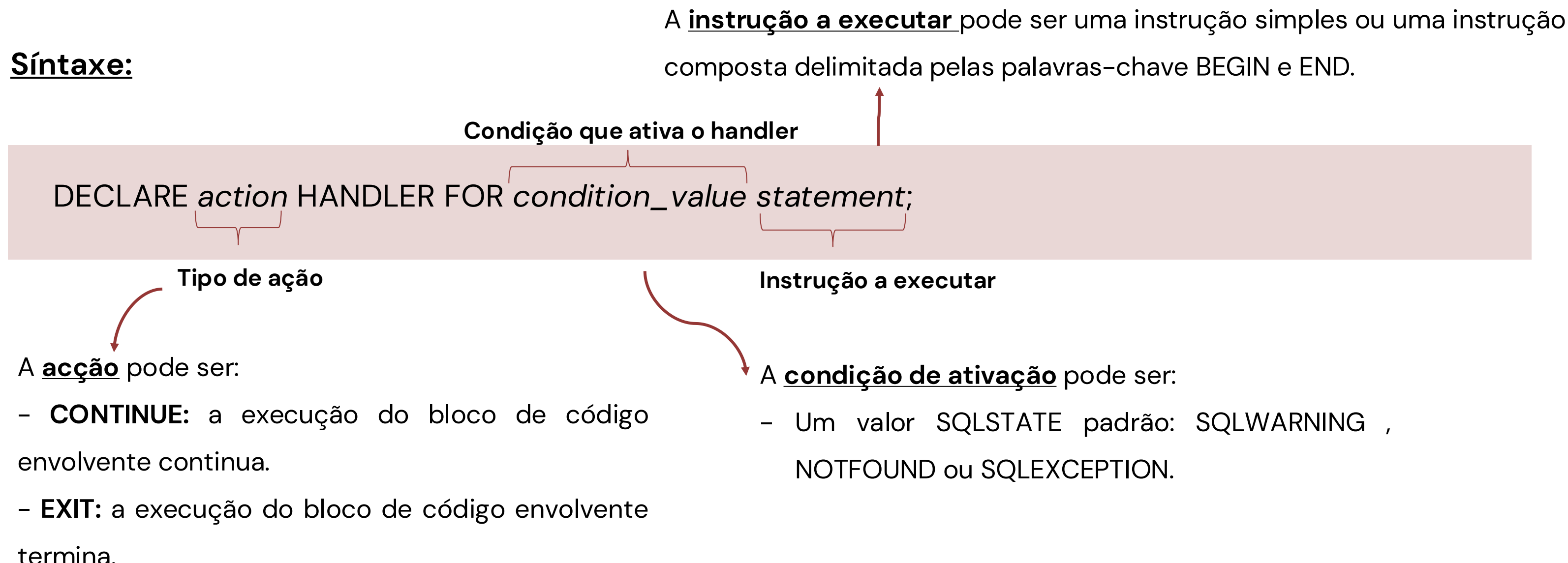
SQL Avançado

➔ Transações

Quando ocorre um erro é importante tratá-lo adequadamente, como continuar ou sair da execução do bloco de código atual e emitir uma mensagem de erro significativa. Para isso usa-se um **handler**, através da instrução DECLARE HANDLER.

Síntaxe:

DECLARE *action* HANDLER FOR *condition_value* *statement*;



The diagram illustrates the syntax of the DECLARE HANDLER statement. The statement is shown in a light pink box: `DECLARE action HANDLER FOR condition_value statement;`. Brackets and arrows identify the components: *action* is labeled 'Tipo de ação' (Type of action); *condition_value* is labeled 'Condição que ativa o handler' (Condition that activates the handler); and *statement* is labeled 'Instrução a executar' (Statement to execute). An additional arrow points from the text 'A instrução a executar pode ser uma instrução simples ou uma instrução composta delimitada pelas palavras-chave BEGIN e END.' to the *statement* part of the syntax.

A instrução a executar pode ser uma instrução simples ou uma instrução composta delimitada pelas palavras-chave BEGIN e END.

Tipo de ação

A acção pode ser:

- **CONTINUE:** a execução do bloco de código envolvente continua.
- **EXIT:** a execução do bloco de código envolvente termina.

Instrução a executar

A condição de ativação pode ser:

- Um valor SQLSTATE padrão: SQLWARNING , NOTFOUND ou SQLEXCEPTION.

SQL Avançado

➔ Transações

Exemplo

```
DELIMITER $$

CREATE PROCEDURE sp_registar_aluguer(
    IN p_inventory_id INT,
    IN p_customer_id INT,
    IN p_staff_id INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;

    INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
    VALUES (NOW(), p_inventory_id, p_customer_id, p_staff_id);

    INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
    VALUES (p_customer_id, p_staff_id, LAST_INSERT_ID(), 4.99, NOW());

    COMMIT;
END$$

DELIMITER ;
```

Bases de Dados

A Linguagem SQL (continuação)

